CODE APEX

THE END IS COMING.

POWERED BY verzii

Team Details

- Team name - null set
- Team leader Name - Anjali Tripathi (Members: Siddharth Tripathi, Pooja Lingwat, Piyush Dadwani)
- Problem Statement (Domain) - Cabinet QC · Emerson PS #03 Project EAGLE EYE

CabinetSight

Photo + drawing in. Marked-up QC report with proof out.
github.com/pegasus1196/QCSight

Problem Statement & Target Need

Every part, label, wire and ferrule is still checked by eye: slow, unproven and easy to miss.

Who is affected

- **Assembly technicians** — learn about mistakes late
- **QC inspectors** — check 100s of items by eye
- **Commissioning teams** — find faults at power-on
- **QC leads / plant** — no evidence trail

The current gap

- Drawing vs cabinet checked by eye at every QC stage
- Hand-filled check sheet: no photo proof, no history
- Generic AI guesses: one false alarm kills trust

Why it matters now

- Hundreds of parts and wires, tight shipping deadlines
- Escaped faults surface at power-on or on site

Faults the tool must catch, with proof

Item	Expected	Found	Result
Power supply PS12	Installed	Missing	**FAIL**
Breaker order	CB41-42-43-44	CB43 ↔ CB44	**FAIL**
Baseplate label	1CBP03	No label	**FAIL**
Ferrule on X3 wire	Fitted	No ferrule	**FAIL**

Example faults from the Emerson problem statement. Each FAIL needs a box on the photo, a zoomed snippet, and must close after re-check.

MUST checks Device presence | Device labels | Wire ferrules

Proposed Solution & Innovation

Photo + drawing in, every fault boxed with proof out. Not sure? It says UNREADABLE, never a false FAIL.

Input

- **Cabinet photo**
- **GA drawing**
- **Wire list**
- **Label list**

CabinetSight engine

1. Find parts, read labels & ferrules
2. Match photo against the drawing
3. Verdict gate: FAIL only on clear proof
4. Re-check after fix: fixed items CLOSED

Output

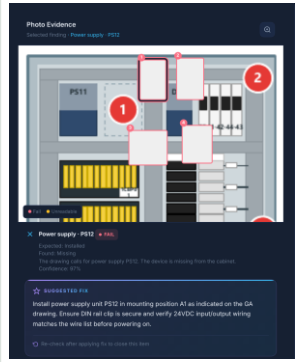


Photo evidence
marker on photo, expected vs found
Suggested fix
what to do, then re-check
QC checklist
PASS / FAIL per item, export

What makes it different

- No false alarms by design
- Proof attached to every FAIL
- New checks plug in, no rewrite

How it decides

PASS | FAIL | UNREADABLE | CLOSED

FAIL only when clearly visible and confident; otherwise UNREADABLE + "retake a close-up". Re-check after fix: fixed items → CLOSED.

Why it's feasible now

- Scope: the 3 MUST checks only
- Working UI prototype already built
- Open-source models, runs offline on a laptop

System Architecture & Tech Stack

Python only where vision needs it. React UI, Docker and security-scanned CI around it.

Tech stack

ROUND 2 BUILD

Frontend

React · TypeScript · Vite · Tailwind · Konva

Backend

Python · FastAPI · Pydantic · pandas · + PyMuPDF

Database

PostgreSQL · + File store

AI / APIs

YOLOv8 · ONNX · PaddleOCR · OpenCV · LLM (text)

Hosting / DevSecOps

Docker · Nginx · Actions · Trivy · + gitleaks

TODAY (prototype) HTML/CSS/JS · FastAPI · OpenCV ORB/SIFT · EasyOCR + Tesseract · RapidFuzz

System architecture (target design)

1. **Inputs** — Photo, GA PDF, wire & label lists
2. **Perceive** — Align photo, detect parts, OCR labels & ferrules
3. **Match** — Drawing ↔ photo: missing, swapped, wrong
4. **Verdict gate** — PASS · FAIL · UNREADABLE, with reason
5. **Report & re-check** — Excel + PDF with proof; re-check open items

Plug-in checks presence · labels · ferrules · colour*

CI gate tests · 0 false alarms · scans · SBOM

On-premise Docker, offline · Prototype: github.com/pegasus1196/QCSight

*All libraries open-source and credited · AI coding assistants used; every member can explain every module · *Wire colour = stretch*

Key Features & UI/UX

One screen: the checklist on the left, the proof on the photo on the right.

Inspection inputs
4 files required, type-checked

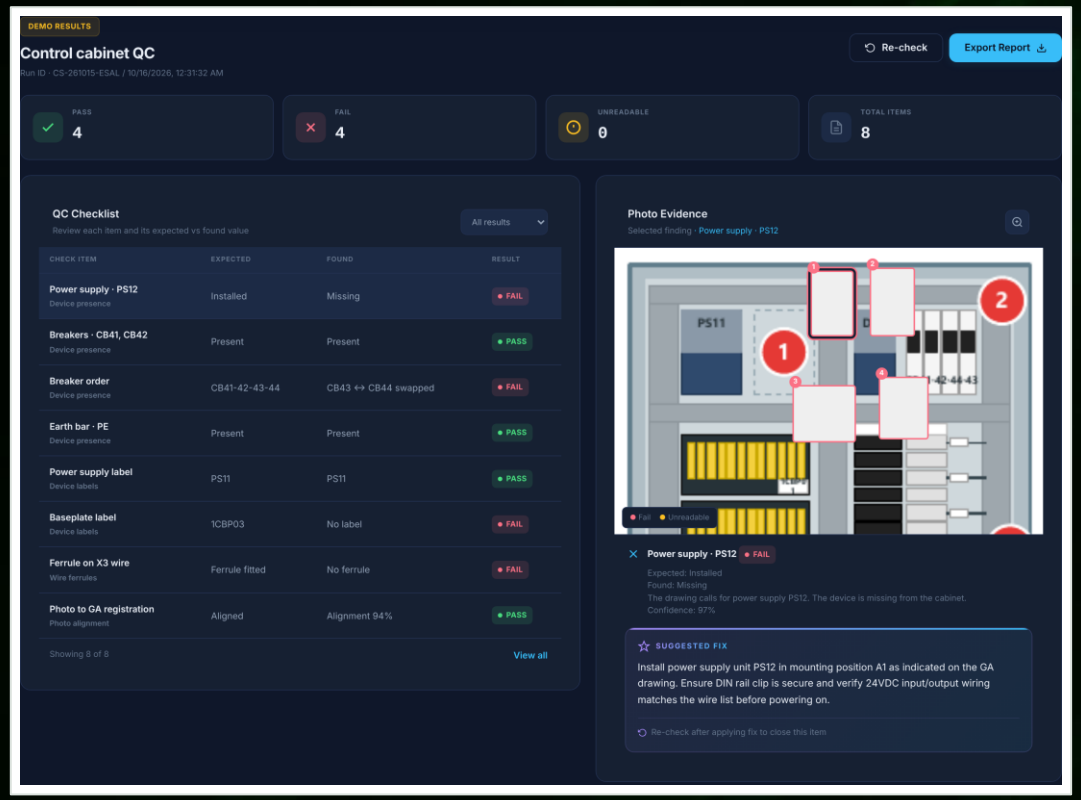
Checks in this run
presence, labels, ferrules, alignment

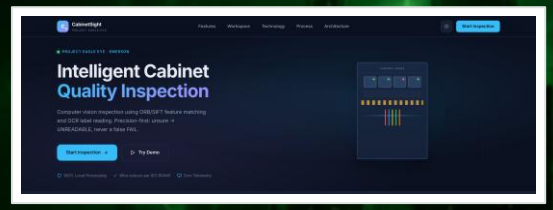
Conservative review
low confidence → UNREADABLE

Photo evidence
marker + suggested fix

Re-check after fix
fixed items marked CLOSED

Export report · Run history
checklist export, runs kept locally





New in this build

- Landing page: 4 inputs · 5 checks · 3 verdicts
- Suggested fix for every FAIL
- Re-check → CLOSED
- Light / dark theme

Working prototype (demo data). Round 2: React + TypeScript build wired to the real engine.

Implementation Roadmap & Scope

What's built today, what's in progress, and what's planned beyond the hackathon

Built today

- HTML/JS UI + FastAPI CV backend
- Re-check flow → CLOSED (demo)
- CSV/print report; suggested fixes
- No-false-FAIL: 5 pytest tests

In progress → Round 2 (18 Oct)

- Cabinet dataset + ground truth
- YOLO detector + PaddleOCR
- GA PDF + Excel parsers, matcher
- React + TypeScript UI

Finale (24 Oct) & beyond

- Real re-check engine; ferrule close-ups
- Docker + DevSecOps CI; wire colour
- Pilot on real plant cabinets

Challenges → our strategy

Few sample photos	→	tiling, augmentation, planted faults
Tiny ferrule text	→	guided close-ups; else UNREADABLE, never FAIL
Unseen cabinets	→	test on cabinets never trained on
Scanned drawings	→	OCR fallback; unknown tags flagged

How we'll prove it

0	false alarms: a correct item is never FAIL
Unseen	cabinets tested with planted faults
< 60 s	per inspection, laptop CPU only

Targets, measured by our eval harness; no merge if false alarms > 0.

Expected Impact & Scalability

Who benefits, how big the opportunity is, and how this grows beyond a demo.

Before → after

Manual QC today	With CabinetSight
Eye-check every item, every stage	**Review only flagged items**
Hand-filled sheet, no evidence	**Boxed proof, audit-ready**
Faults found at power-on or site	**Caught at the build bench**
Re-check = walk the cabinet again	**Re-check after fix → CLOSED**

PROOF TODAY **5 pytest tests enforce the no-false-FAIL rule**

Who benefits

- **QC inspectors**
  faster, consistent checks
- **Panel builders**
  fewer late-found faults
- **Commissioning teams**
  clear fix lists
- **Plant QA**
  evidence trail for audits

How it scales

Pilot line
one cabinet family

→ **Every QC stage**
assembly · power-on · final

→ **Any panel shop**
cabinets built from drawings

→ **New plug-ins**
earthing, shrouds, AR

VISHWAKARMA INSTITUTES VI | AISF ARTIFICIAL INTELLIGENCE STUDENT FORUM

VCA

CODE APEX

BUILD BEFORE IT DOES.

POWERED BY verzii

THANK YOU

Team null set · CabinetSight · github.com/pegasus1196/QCSight